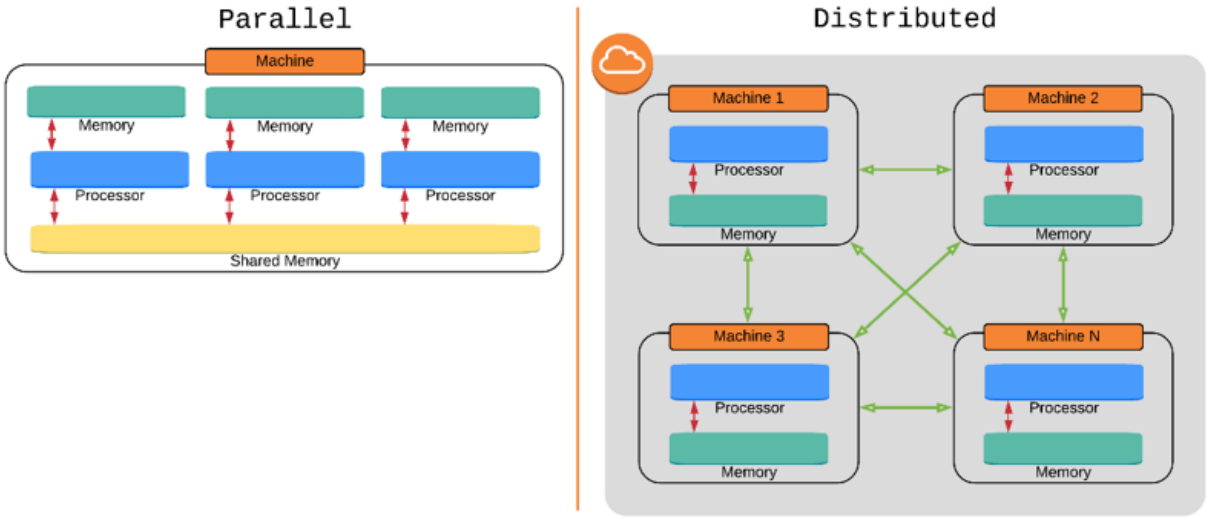


Figure 13.1 A schematic view of the difference between parallel and distributed computing models.

13.1.2 Parallelizing k-means

Let's now get more specific, in order to answer this question: can we make k-means a parallel algorithm?

Looking at each step of the algorithm separately will help us "divide and conquer" the problem. Please refer to section 12.2 for the description and implementation of k-means.

- The first step is **initialization**, creating an initial guess for the centroids. If this is done completely at random, this step is independent on the dataset and its running time is only proportional to the number k of clusters; therefore, the fully randomized version is not worth parallelizing. Parallelization could be necessary, instead, when points are drawn independently from a distribution "without replacement": we'll see how to distribute this step in section 13.3.2.